

CSCE 775 RL Quiz 1 Reference

Sutton & Barto: Ch1, Ch3, Ch4

J.C. Vaught

1 Introduction to Reinforcement Learning

1.1 Definition

Reinforcement Learning: Learning what to do—mapping situations to actions to maximize a numerical reward signal.

1.2 Key Features

- Trial-and-error search for good behavior
 - Delayed reward (actions affect not just immediate but also subsequent rewards)
 - Sequential decision making under uncertainty
- ### 1.3 RL vs Other Learning Paradigms
- **Supervised learning:** Learning from labeled examples provided by external supervisor
 - **Unsupervised learning:** Finding structure in unlabeled data
 - **Reinforcement learning:** Learning from interaction to maximize reward signal; no labeled examples of correct/incorrect actions, must discover through trial-and-error

1.4 Exploration vs Exploitation

Exploration-Exploitation Tradeoff: Agent must exploit known good actions to obtain reward, but also explore new actions to discover potentially better ones. Neither pure exploration nor pure exploitation solves the task.

1.5 Four Elements of RL

1. **Policy** π : Mapping from states to probabilities of selecting each action; defines agent's behavior
2. **Reward Signal** R_t : Single number sent to agent at each time step; defines goal; agent seeks to maximize total reward
3. **Value Function** $v_\pi(s)$ or $q_\pi(s, a)$: Total reward expected in the future from a state (or state-action pair); guides action selection
4. **Model** (optional): Mimics environment behavior; allows planning by considering future situations before experiencing them

1.6 Model-Based vs Model-Free

- **Model-based methods:** Use a model of the environment for planning
- **Model-free methods:** Learn purely from trial-and-error experience without explicit model

1.7 Temporal-Difference (TD) Learning

Basic TD update rule:

$$V(S_t) \leftarrow V(S_t) + \alpha [V(S_{t+1}) - V(S_t)] \quad (1)$$

where α is the learning rate (step size).

TD combines ideas from Monte Carlo (learning from experience) and Dynamic Programming (bootstrapping from estimates).

2 Finite Markov Decision Processes

2.1 Agent-Environment Interface

Agent and environment interact in discrete time steps: $t = 0, 1, 2, 3, \dots$

Trajectory: $S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots$

At each time t :

- Agent observes state $S_t \in \mathcal{S}$
- Selects action $A_t \in \mathcal{A}(s)$
- Receives reward $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$
- Transitions to next state S_{t+1}

2.2 Dynamics Function

Complete probability distribution defining MDP dynamics:

$$p(s', r | s, a) = \Pr\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\} \quad (2)$$

for all $s', s \in \mathcal{S}, r \in \mathcal{R}, a \in \mathcal{A}(s)$.

Must satisfy: $\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r | s, a) = 1$ for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$.

2.3 Derived Quantities

State-transition probabilities:

$$p(s' | s, a) = \Pr\{S_t = s' \mid S_{t-1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r | s, a) \quad (3)$$

Expected reward for state-action pair:

$$r(s, a) = \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r | s, a) \quad (4)$$

Expected reward for state-action-next-state triple:

$$r(s, a, s') = \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a, S_t = s'] = \sum_{r \in \mathcal{R}} r \frac{p(s', r | s, a)}{p(s' | s, a)} \quad (5)$$

2.4 Markov Property

Markov Property: The future depends only on the current state, not on the history.

$$\Pr\{S_t = s', R_t = r \mid S_0, A_0, R_1, \dots, S_{t-1}, A_{t-1}\} = p(s', r | S_{t-1}, A_{t-1}) \quad (6)$$

A state signal has the Markov property if it includes all relevant information from history.

2.5 Returns

Episodic tasks (terminate in finite steps at terminal state):

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T \quad (7)$$

where T is the terminal time step.

Continuing tasks (no terminal state, with discounting):

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \quad (8)$$

where $0 \leq \gamma \leq 1$ is the discount rate.

Recursive form:

$$G_t = R_{t+1} + \gamma G_{t+1} \quad (9)$$

Special case (constant reward +1 forever):

$$G_t = \sum_{k=0}^{\infty} \gamma^k = \frac{1}{1 - \gamma} \quad (10)$$

2.6 Policies

Policy π : Mapping from states to probability distributions over actions.

$$\pi(a | s) = \Pr\{A_t = a \mid S_t = s\} \quad (11)$$

Deterministic policy: $\pi(s) = a$ (single action for each state).

2.7 Value Functions

State-value function for policy π :

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right] \quad (12)$$

Action-value function (Q-function) for policy π :

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right] \quad (13)$$

Relationship between value functions:

$$v_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a | s) q_\pi(s, a) \quad (14)$$

2.8 Bellman Equations

Bellman equation for v_π :

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_\pi(s')] \quad (15)$$

Expanded form:

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \quad (16)$$

Bellman equation for q_π :

$$q_\pi(s, a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \sum_{a'} \pi(a'|s') q_\pi(s', a') \right] \quad (17)$$

Expanded form:

$$q_\pi(s, a) = \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a] \quad (18)$$

2.9 Optimal Value Functions

Optimal state-value function:

$$v_*(s) = \max_\pi v_\pi(s) \quad \text{for all } s \in \mathcal{S} \quad (19)$$

Optimal action-value function:

$$q_*(s, a) = \max_\pi q_\pi(s, a) \quad \text{for all } s \in \mathcal{S}, a \in \mathcal{A}(s) \quad (20)$$

Relationship:

$$v_*(s) = \max_{a \in \mathcal{A}(s)} q_*(s, a) \quad (21)$$

$$q_*(s, a) = \sum_{s',r} p(s',r|s,a) [r + \gamma v_*(s')] \quad (22)$$

2.10 Bellman Optimality Equations

Bellman optimality equation for v_* :

$$v_*(s) = \max_{a \in \mathcal{A}(s)} \sum_{s',r} p(s',r|s,a) [r + \gamma v_*(s')] \quad (23)$$

Bellman optimality equation for q_* :

$$q_*(s, a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \max_{a' \in \mathcal{A}(s')} q_*(s', a') \right] \quad (24)$$

2.11 Optimal Policy

Optimal policy (greedy with respect to q_*):

$$\pi_*(s) = \arg \max_{a \in \mathcal{A}(s)} q_*(s, a) \quad (25)$$

Alternative form (greedy with respect to v_* using one-step lookahead):

$$\pi_*(s) = \arg \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma v_*(s')] \quad (26)$$

Any policy that is greedy with respect to v_* or q_* is an optimal policy.

3 Dynamic Programming

DP methods require perfect model of environment: $p(s', r|s, a)$.

3.1 Policy Evaluation (Prediction)

Goal: Compute v_π for a given policy π .

Iterative update (until convergence):

$$V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V_k(s')] \quad (27)$$

for all $s \in \mathcal{S}$.

Convergence criterion: Stop when $\max_{s \in \mathcal{S}} |V_{k+1}(s) - V_k(s)| < \theta$ for small threshold $\theta > 0$. As $k \rightarrow \infty$, $V_k \rightarrow v_\pi$ under Bellman equation constraints.

Implementation variants:

- **Two-array (Jacobi):** Use old values for all states in sweep
- **In-place (Gauss-Seidel):** Use new values as soon as available; typically faster convergence

Algorithm 1: Iterative Policy Evaluation

Input: Policy π , threshold $\theta > 0$

Initialize $V(s) = 0$ for all $s \in \mathcal{S}^+$

repeat

$\Delta \leftarrow 0$

for each $s \in \mathcal{S}$ **do**

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

end for

until $\Delta < \theta$

Output: $V \approx v_\pi$

3.2 Policy Improvement

Policy improvement theorem: Let π and π' be deterministic policies such that for all $s \in \mathcal{S}$:

$$q_\pi(s, \pi'(s)) \geq v_\pi(s) \quad (28)$$

Then $v_{\pi'}(s) \geq v_\pi(s)$ for all $s \in \mathcal{S}$.

Greedy policy improvement:

$$\pi'(s) = \arg \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')] \quad (29)$$

This one-step lookahead constructs a greedy policy with respect to current value function.

If $v_{\pi'} = v_\pi$, then both π and π' are optimal (Bellman optimality equation satisfied).

3.3 Policy Iteration

Algorithm: Alternate between policy evaluation and policy improvement.

Algorithm 2: Policy Iteration

Initialize $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily

repeat

POLICY EVALUATION:

repeat

$\Delta \leftarrow 0$

for each $s \in \mathcal{S}$ **do**

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

end for

until $\Delta < \theta$

POLICY IMPROVEMENT:

$policy_stable \leftarrow true$

for each $s \in \mathcal{S}$ **do**

$old_action \leftarrow \pi(s)$

$\pi(s) \leftarrow \arg \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

if $old_action \neq \pi(s)$ **then** $policy_stable \leftarrow false$

end for

if $policy_stable$ **then return** $V \approx v_*$, $\pi \approx \pi_*$

until π stable

Policy iteration converges to π_* and v_* in finite iterations (for finite MDPs).

3.4 Value Iteration

Key idea: Combine truncated policy evaluation (one sweep) with policy improvement.

Update rule (turn Bellman optimality equation into assignment):

$$V_{k+1}(s) = \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V_k(s')] \quad (30)$$

Algorithm 3: Value Iteration**Input:** Threshold $\theta > 0$ Initialize $V(s)$ arbitrarily, e.g., $V(s) = 0$ for all $s \in \mathcal{S}^+$ **repeat** $\Delta \leftarrow 0$ **for each** $s \in \mathcal{S}$ **do** $v \leftarrow V(s)$ $V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$ $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ **end for****until** $\Delta < \theta$

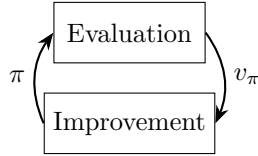
OUTPUT DETERMINISTIC POLICY:

for each $s \in \mathcal{S}$ **do** $\pi(s) \leftarrow \arg \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$ **end for****Output:** $V \approx v_*$ and $\pi \approx \pi_*$

Faster than policy iteration for some problems; does not require explicit policy during iteration.

3.5 Generalized Policy Iteration (GPI)

GPI: General idea of interacting processes of policy evaluation and policy improvement.



Both processes stabilize when:

$$v_\pi(s) = \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma v_\pi(s')] \quad (31)$$

which is the Bellman optimality equation, implying $v_\pi = v_*$ and $\pi = \pi_*$.

Key insight: Almost all RL methods are forms of GPI.

3.6 Asynchronous Dynamic Programming

Asynchronous DP: Update states in any order, using whatever values are available.

- Don't require systematic sweeps of entire state set
 - Can update individual states multiple times before updating others
 - Still converges if all states continue to be updated
 - Allows focusing computation on relevant states
- Examples: prioritized sweeping, real-time dynamic programming.

3.7 Bootstrapping

Bootstrapping: Updating estimates based on other estimates (rather than complete returns).

DP methods bootstrap: use $V(s')$ to update $V(s)$ before $V(s')$ has converged.

3.8 Computational Complexity

DP is polynomial in number of states $|\mathcal{S}|$ and actions $|\mathcal{A}|$.

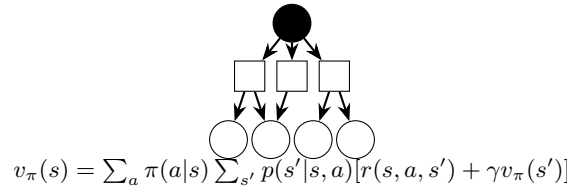
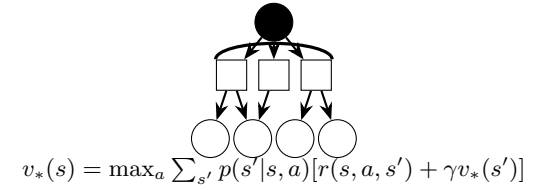
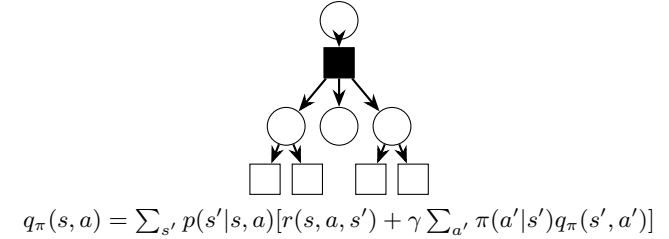
For k binary state variables: $|\mathcal{S}| = 2^k$

Number of deterministic policies: $|\mathcal{A}|^{|\mathcal{S}|}$ (exponentially large)

DP is exponentially faster than exhaustive policy search.

Even with large state spaces, DP can be practical if we can:

- Use asynchronous methods
- Use function approximation
- Focus on relevant states

4 Backup Diagrams**4.1 v_π Backup Diagram****4.2 v_* Backup Diagram****4.3 q_π Backup Diagram****4.4 q_* Backup Diagram**